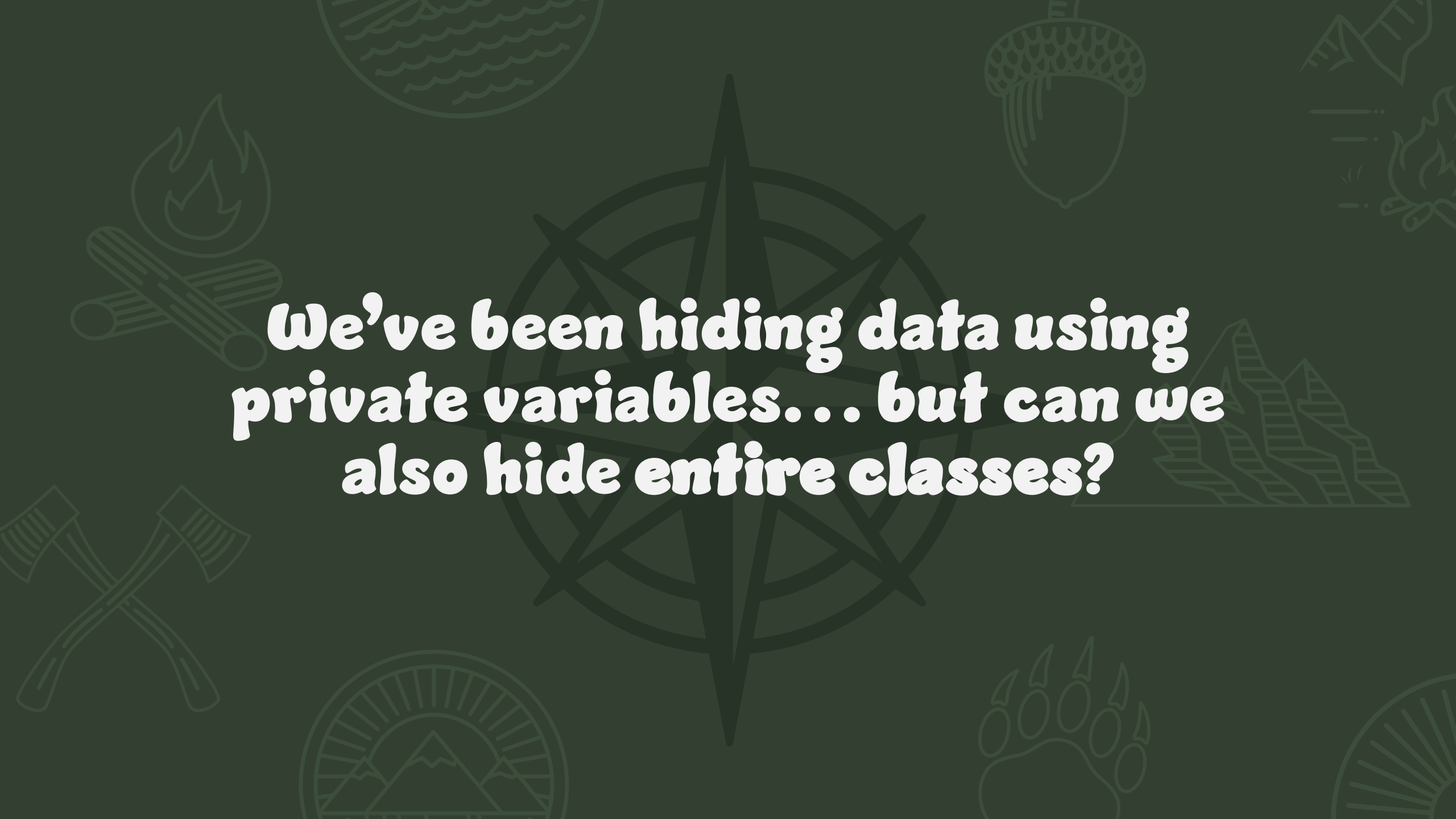




We've been hiding data using private variables... but can we also hide entire classes?

Inner classes

AN *INNER CLASS* IS A CLASS DEFINED
INSIDE ANOTHER CLASS

USED WHEN IT IS TIGHTLY COUPLED
WITH THE OUTER CLASS.



Why Inner Classes?

- **Encapsulation** → hide implementation ✓
- **Inner Class** → hide *entire class* ✓✓
- “When a class is only useful to its outer class, we keep it inside.”

What will be the output of the following?

```
class OuterClass {  
    int x = 10;  
  
    class InnerClass {  
        int y = 5;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        OuterClass myOuter = new OuterClass();  
        OuterClass.InnerClass myInner = myOuter.new InnerClass();  
        System.out.println(myInner.y + myOuter.x);  
    }  
}
```

What is this strange line doing?

```
OuterClass.InnerClass myInner = myOuter.new  
InnerClass();
```



Explanation

Outer Class -> `OuterClass myOuter = new OuterClass();`

Inner class -> `OuterClass.InnerClass myInner = myOuter.new InnerClass();`

`OuterClass.InnerClass` → tells compiler: *Inner class belongs to OuterClass*

`myOuter.new InnerClass()` → inner class object needs **outer class object**

Key Idea: *Inner class cannot exist without outer class*

Access Relationship



INNER CLASS CAN ACCESS OUTER
CLASS MEMBERS



OUTER CLASS DOES NOT DIRECTLY
ACCESS INNER CLASS MEMBERS

Inner class accessing outer class member

```
class OuterClass {  
    int x = 10;  
  
    class InnerClass {  
        int y = 5;  
        int z = x + y;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        OuterClass myOuter = new OuterClass();  
        OuterClass.InnerClass myInner = myOuter.new InnerClass();  
        System.out.println(myInner.z);  
        System.out.println(myInner.y + myOuter.x);  
    }  
}
```

What about this?

```
class OuterClass {  
    int x = 10;  
    int k = y + x;  
  
    class InnerClass {  
        int y = 5;  
        int z = x + y;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        OuterClass myOuter = new OuterClass();  
        OuterClass.InnerClass myInner = myOuter.new InnerClass();  
        System.out.println(myInner.z);  
        System.out.println(myInner.y + myOuter.x);  
    }  
}
```

Explanation

- `y` belongs to **InnerClass**
- OuterClass has **no direct access** to it
- No inner object = no access
- Inner class lives inside Outer... but Outer does NOT live inside Inner.

✓ Works (Inner → Outer)

⟨/⟩ Java

```
int z = x + y;
```

✗ Fails (Outer → Inner)

⟨/⟩ Java

```
int k = y + x;
```

Solution

```
class OuterClass {
    int x = 10;
    OuterClass() {
        InnerClass obj = new InnerClass();
        int k = obj.y + x;
    }

    class InnerClass {
        int y = 5;
        int z = x + y;
    }
}

public class Main {
    public static void main(String[] args) {
        OuterClass myOuter = new OuterClass();
        OuterClass.InnerClass myInner = myOuter.new InnerClass();
        System.out.println(myInner.z);
        System.out.println(myInner.y + myOuter.x);
    }
}
```

Explanation

- we need obj.y
- Because inner class is treated like a **separate object**
- Inner gets a free pass to Outer... Outer must ask permission (object) to access Inner.
- Inner class object **always needs an outer class object**

Even though you wrote:

```
</> Java  
new InnerClass()
```

Java internally treats it like:

```
"Create InnerClass for this specific OuterClass object"
```

So it's actually:

```
</> Java  
this.new InnerClass()
```

Inner class object always needs an outer class object

That's why:

Java

```
OuterClass.InnerClass obj = myOuter.new InnerClass();
```

or inside same class:

Java

```
InnerClass obj = new InnerClass();
```

both work (because `this` is implied)

Action	Meaning
<code>new OuterClass()</code>	creates outer object
<code>new InnerClass()</code>	creates inner object tied to that outer
<code>obj.y</code>	access inner member

Making inner class private

```
class OuterClass {  
    int x = 10;  
  
    private class InnerClass {  
        int y = 5;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        OuterClass myOuter = new OuterClass();  
        OuterClass.InnerClass myInner = myOuter.new InnerClass();  
        System.out.println(myInner.y + myOuter.x);  
    }  
}
```

Making inner class static

```
class OuterClass {  
    int x = 10;  
  
    static class InnerClass {  
        int y = 5;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        OuterClass.InnerClass myInner = new OuterClass.InnerClass();  
        System.out.println(myInner.y);  
    }  
}
```

Making inner class static

- **Normal Inner Class:** “Belongs to an object of OuterClass”
- **Static Nested Class:** Belongs to OuterClass itself, not its objects”

✗ Before (Inner Class - dependent)

</> Java

```
myOuter.new InnerClass();
```

✓ Needs outer object

⚡ Now (Static Nested Class - independent)

</> Java

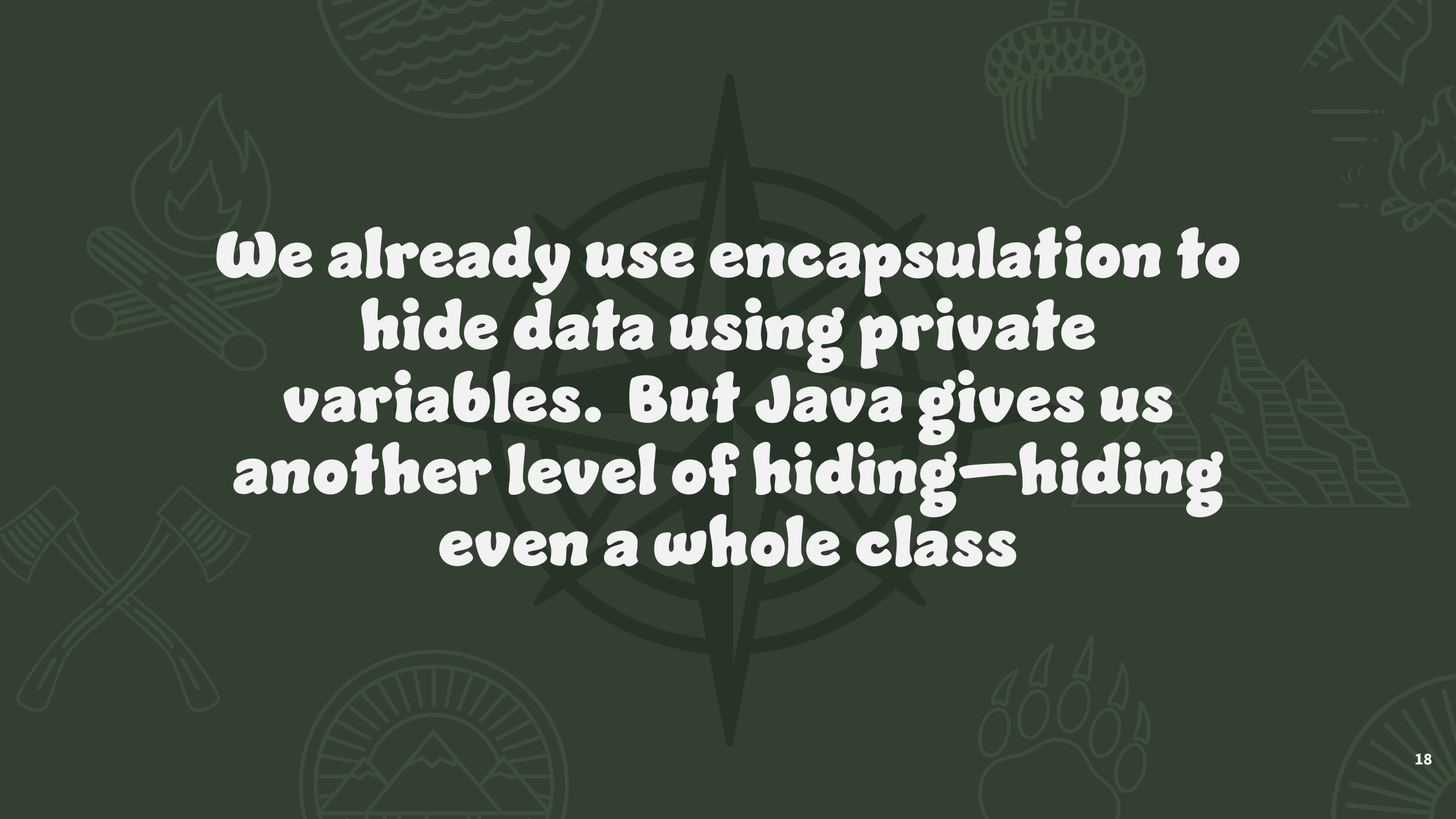
```
new OuterClass.InnerClass();
```

✓ No outer object required



What is use of it?





**We already use encapsulation to
hide data using private
variables. But Java gives us
another level of hiding—hiding
even a whole class**



Remember Encapsulation

- 
- ✓ Data hiding (variables)
 - ✓ Controlled access (methods)

What if not only data, but even helper logic should be hidden?

That's where **inner classes + encapsulation combine.**



Example

- Data Encapsulation: private int x;
- **Logic Encapsulation:**
private class InnerClass
- Controlled Access: public int calculate()

```
class OuterClass {  
  
    private int x = 10;  
  
    // fully hidden implementation  
    private class InnerClass {  
        int y = 5;  
  
        int getResult() {  
            return x + y;  
        }  
    }  
  
    public int calculate() {  
        InnerClass obj = new InnerClass();  
        return obj.getResult();  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
  
        OuterClass obj = new OuterClass();  
  
        System.out.println(obj.calculate());  
    }  
}
```

Key Insight

Encapsulation hides data, but inner classes hide implementation.

Without inner class:

- Helper logic is exposed or scattered

With inner class:

- Helper logic is **fully contained inside the class**

What is hidden?

Data (x)

Logic (InnerClass)

Internal computation

How?

private variable

private inner class

wrapped in calculate()

Real-life examples

Robot idea:

Sensor → inside Robot

Car idea:

Engine → inside Car

Music idea:

Playlist → inside Player

Bank idea:

Transaction → inside Account





Robot Example (Sensor inside Robot)

```
class Robot {  
  
    private String name = "Miro-e";  
  
    class Sensor {  
        void read() {  
            System.out.println(name + " sensor reading data...");  
        }  
    }  
  
    void start() {  
        Sensor s = new Sensor();  
        s.read();  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Robot r = new Robot();  
        r.start();  
    }  
}
```

Car Example (Engine inside Car)

```
class Car {  
  
    class Engine {  
        void run() {  
            System.out.println("Engine is running...");  
        }  
    }  
  
    void startCar() {  
        Engine e = new Engine();  
        e.run();  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Car c = new Car();  
        c.startCar();  
    }  
}
```

Music Player Example (Playlist inside Player)

```
class MusicPlayer {  
  
    class Playlist {  
        void play() {  
            System.out.println("Playing playlist...");  
        }  
    }  
  
    void start() {  
        Playlist p = new Playlist();  
        p.play();  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        MusicPlayer m = new MusicPlayer();  
        m.start();  
    }  
}
```

Bank Account Example (Transaction inside Account)

```
class BankAccount {  
  
    private int balance = 1000;  
  
    class Transaction {  
        void deposit(int amount) {  
            balance += amount;  
            System.out.println("New Balance: " + balance);  
        }  
    }  
  
    void doTransaction() {  
        Transaction t = new Transaction();  
        t.deposit(500);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        BankAccount b = new BankAccount();  
        b.doTransaction();  
    }  
}
```